Wire RC

Every buffering decision starts from how much resistance and capacitance a wire segment carries

The idea

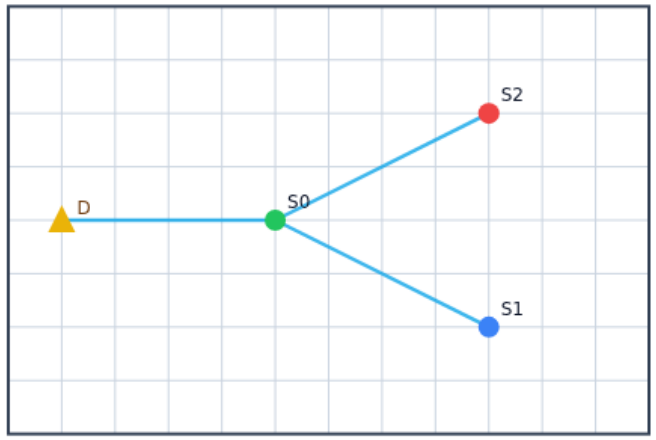
- Wire_rc of length L returns R equals L times r_per and C equals L times c_per
- Elmore delay uses R times C_self over two plus C_load

Net topology

WIRE RC

Net topology

Step 1 / 5 · net · module01-01-wire-rc



Explanation

Driver D feeds S0; S0 fans to S1 and S2 on the 12×8 chip.

- Three sinks
- Three edges

```
3 edges
```

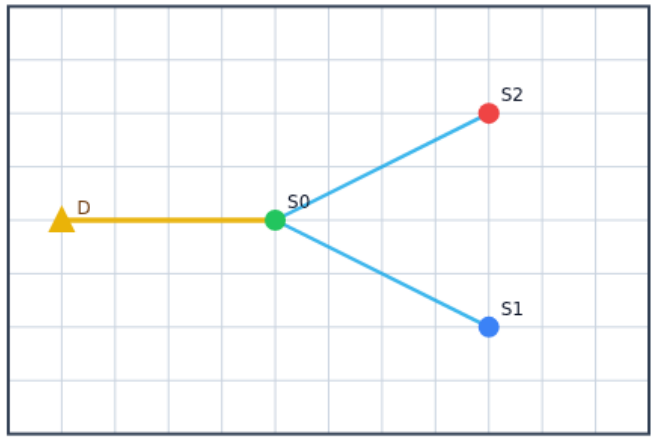
Net topology

Edge e0 length

WIRE RC

Edge e0 length

Step 2 / 5 · e0 · module01-01-wire-rc



Explanation

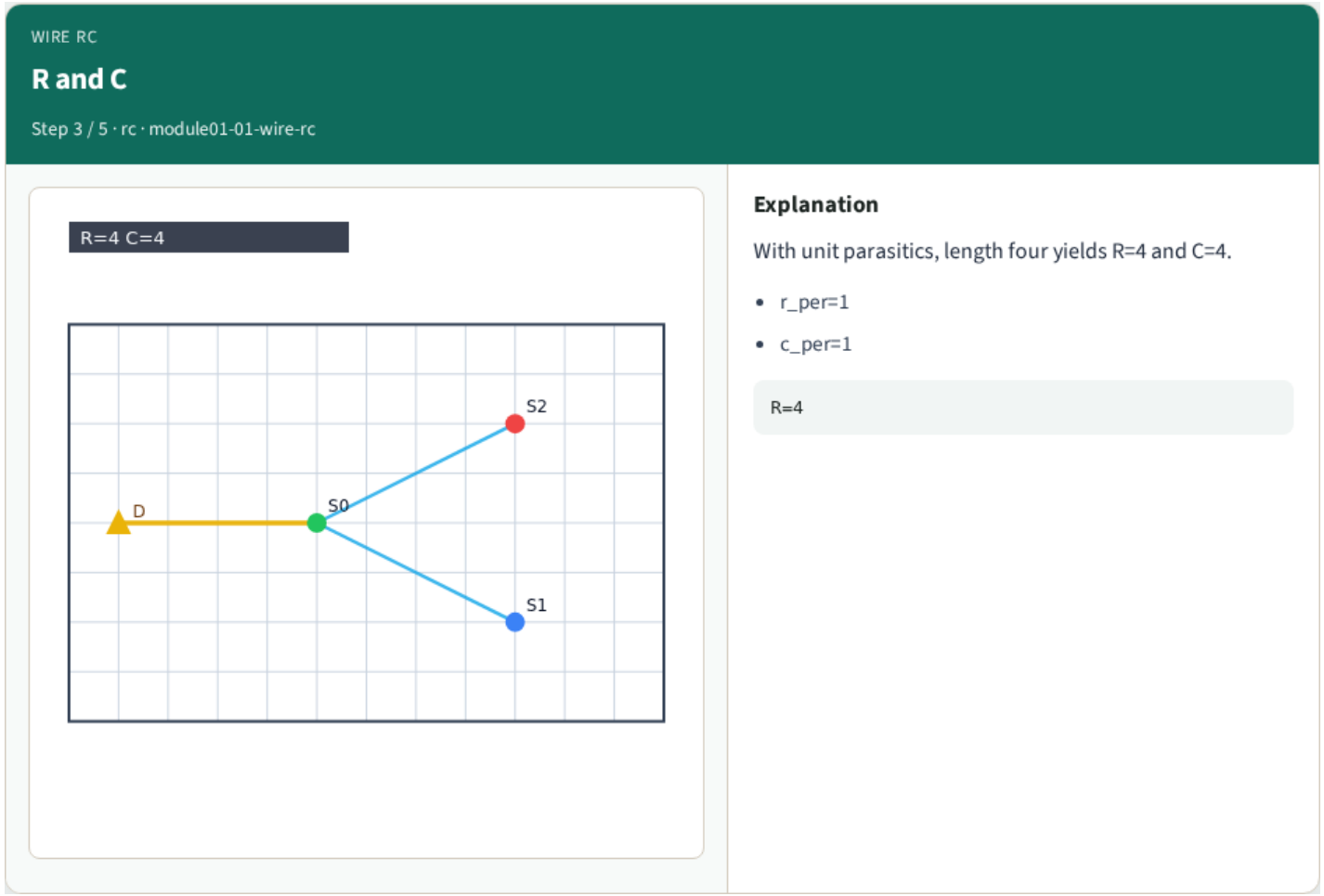
D → S0 is four units Manhattan on the starter.

- Manhattan
- Unit parasitics

```
len≈4
```

Edge e0 length

R and C


WIRE RC
R and C
Step 3 / 5 · rc · module01-01-wire-rc
R=4 C=4
S2
D
S0
S1
Explanation
With unit parasitics, length four yields R=4 and C=4.
r_per=1
c_per=1
R=4

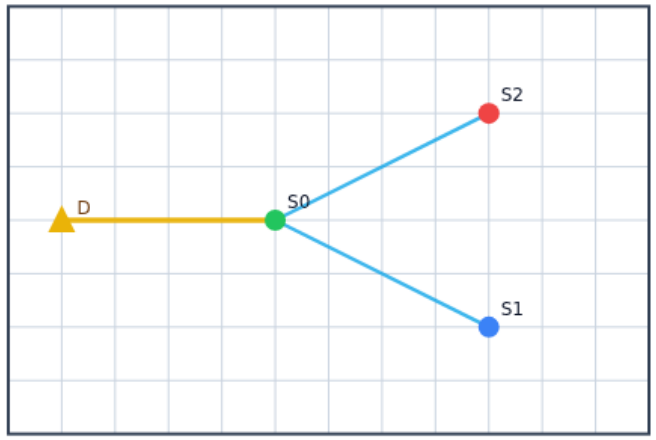
R and C

Elmore

WIRE RC

Elmore

Step 4 / 5 · elmore · module01-01-wire-rc



Explanation

Delay ≈ R·(C/2 + Cload) on a lumped segment.

- Lumped model
- Toy

```
Elmore
```

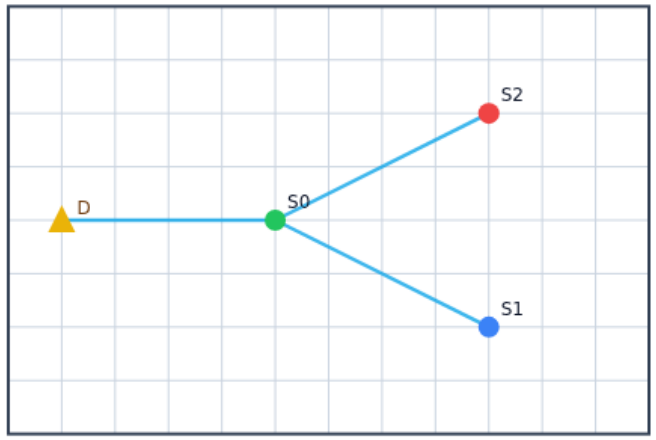
Elmore

Why it matters

WIRE RC

Why it matters

Step 5 / 5 · why · module01-01-wire-rc



Explanation

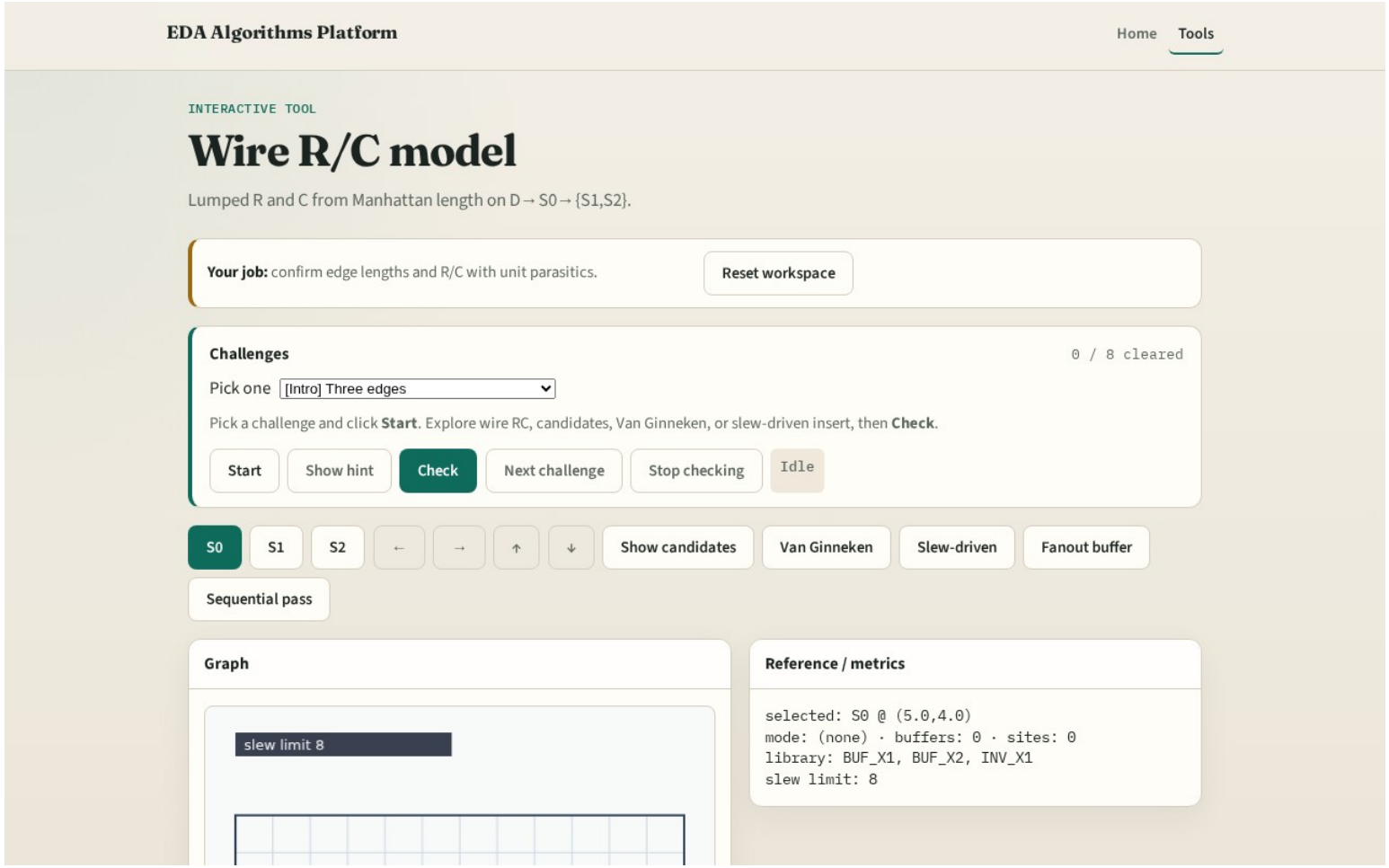
Longer edges dominate slew and delay — candidates appear next.

- RC drives insert
- Next lab

```
Slew budget
```

Why it matters

Browser lab track



Browser lab starter

Implement track

- Call `wire_rc(length)` and `elmore_delay` in `common/`
- Match GOLDENS.wireRcLen4R

Pitfalls

- Using Euclidean length
- Forgetting C_load in Elmore
- Mixing per-unit R with total R

Your turn

- Next: slew budgets on those RC segments